CS-1103
Final Report
University Database Management System

Abstract

This project presents the design and implementation of a University Database Management System using MySQL and Java with JDBC connectivity. The system manages academic entities such as students, instructors, departments, courses, and enrollments. It supports essential database operations including insertion, retrieval, updating, and deletion of data through a command-line interface. The project demonstrates the application of database concepts such as ER modeling, relational schema design, and referential integrity. The integration of Java and JDBC allows users to interact with the database efficiently without directly writing SQL queries.

Introduction

The purpose of this project is to design and implement a University Database Management System using relational database concepts. The system simulates how a university manages academic data such as students, instructors, departments, courses, and enrollments.
This project applies key concepts learned throughout the course, including entity-relationship modeling, schema design, SQL operations, and database connectivity. The system allows efficient storage, retrieval, and management of academic information and provides a practical understanding of how databases are used in real-world applications.

The System

The University Database Management System was developed using MySQL and Java with JDBC connectivity. The system is based on an ER diagram that represents the structure of the database and relationships between entities.

Database Design (ER Diagram)

The system includes several main entities such as Student, Instructor, Department, Course, Section, Classroom, and Time_Slot. Each entity contains attributes relevant to the university system. For example, the student entity stores ID, name, total credits, and department information, while the Instructor entity stores ID, name, salary, and department.
Relationships between entities are also defined. A Student is associated with an Instructor through the advisor relationship. Students enroll in course sections through the takes relationship, and instructors teach sections through the teaches relationship. Sections are linked to classrooms and time slots to represent where and when classes are held.

DDL Implementation (Schema Design)

The ER diagram was converted into relational tables using SQL DDL statements. Each entity was implemented as a table with appropriate primary keys. For example, the Student and Instructor tables use unique IDs as primary keys, while the Classroom table uses a composite key (building, room_number).
Foreign keys were used to maintain relationships between tables. For instance, the Student, Instructor, and Course tables include department references, while the Section table references the Course table.

Relationship tables such as takes, teaches, and advisor were created to represent interactions between entities.
Constraints such as ON DELETE CASCADE and ON DELETE SET NULL were used to maintain referential integrity and ensure consistent data.

Sample Data

Sample data was inserted into all tables using SQL INSERT statements. The dataset includes multiple departments, students, instructors, courses, and sections across different semesters.
Relationship tables were also populated to represent real-world interactions. For example, students are enrolled in multiple courses, instructors are assigned to teach sections, and prerequisite relationships are defined between courses.
This data allows testing of the database and demonstrates how the system handles real academic scenarios.

JDBC Integration and Application

The system includes a Java-based application that connects to the database using JDBC. A Database Connection class is used to establish and manage the connection to the MySQL database.
The application provides a command-line interface (CLI) that allows users to interact with the system. Through this interface, users can perform CRUD operations such as adding, viewing, updating, and deleting records.
Additional features include search functionality and report generation. Users can search for students, instructors, and courses based on different criteria. The system also demonstrates advanced queries such as calculating instructor income tax and generating reports.
The application follows an object-oriented design and uses multiple classes to manage different components of the system.

Conclusion

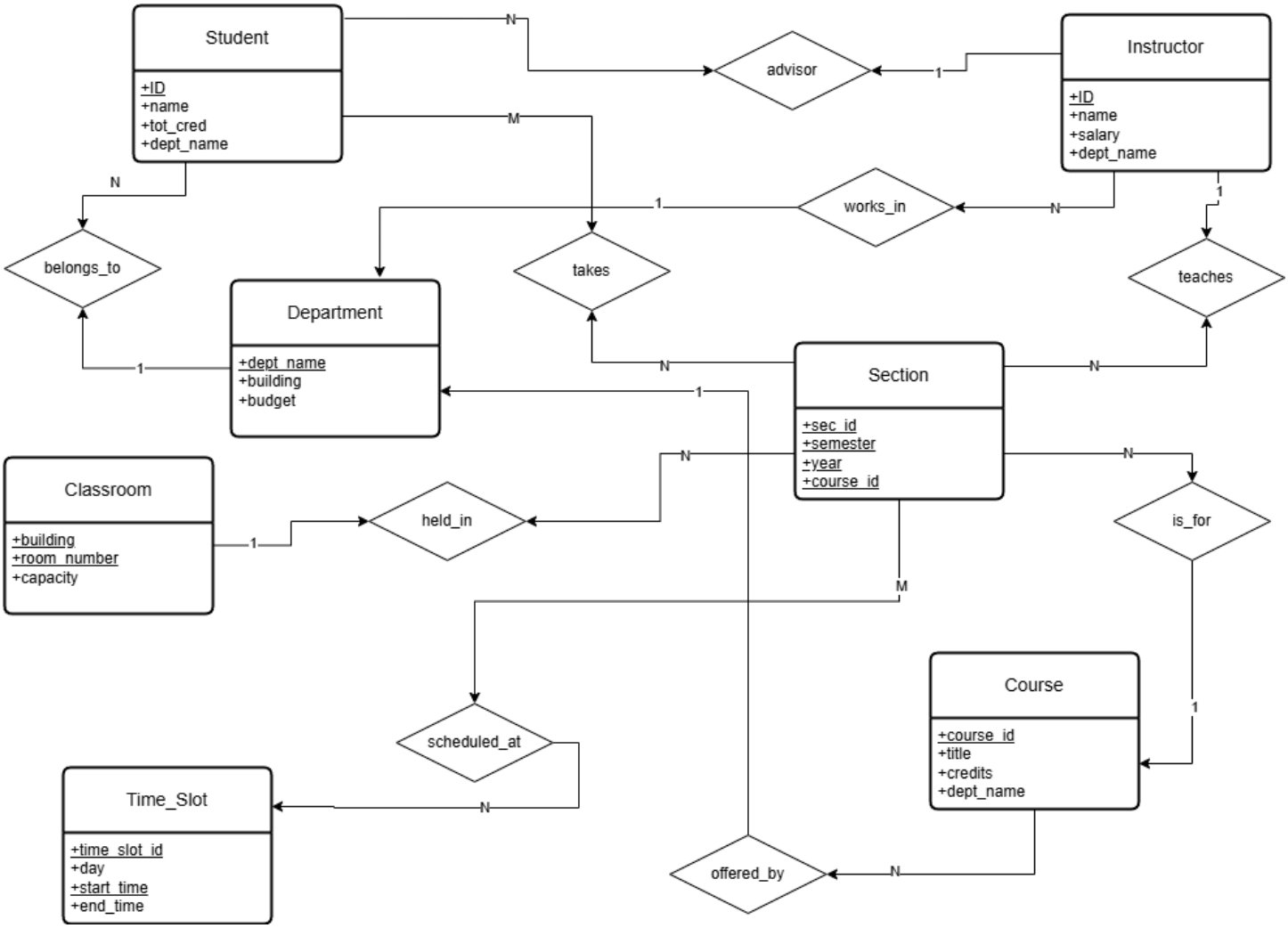
This project provided valuable hands-on experience in designing and implementing a relational database system. It reinforced key concepts such as ER modeling, SQL schema design, and database connectivity using JDBC.
One of the main challenges faced was ensuring stable database connectivity across different environments and handling errors during JDBC implementation. Maintaining data consistency and resolving SQL-related issues also required careful debugging.
Despite these challenges, the project successfully demonstrates a functional university database system with a user-friendly interface. Future improvements could include developing a web-based interface and adding advanced reporting features.

Bibliography

1. Oracle JDBC Documentation – https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/
2. W3Schools SQL Tutorial – https://www.w3schools.com/sql/
3. GeeksforGeeks – JDBC in Java – https://www.geeksforgeeks.org/jdbc-in-java/
4. Course Lecture Notes (CS1103 – Intro to Database)

Appendix

ER Diagram



SQL DDL Statements

schema.sql

```
-- Student entity (Foreign Key)
CREATE TABLE student (
   ID VARCHAR(10) PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   dept_name VARCHAR(50),
   FOREIGN KEY (dept_name) REFERENCES department(dept_name) ON DELETE SET NULL
);

-- Section entity (Composite Key)
CREATE TABLE section (
   sec_id VARCHAR(10),
   semester VARCHAR(10),
   year INT,
   course_id VARCHAR(10),
   PRIMARY KEY (sec_id, semester, year, course_id)
);

-- Takes relationship (Many-to-Many)
CREATE TABLE takes (
   student_id VARCHAR(10),
   sec_id VARCHAR(10),
   semester VARCHAR(10),
   year INT,
   course_id VARCHAR(10),
   grade VARCHAR(2),
   PRIMARY KEY (student_id, sec_id, semester, year, course_id),
   FOREIGN KEY (student_id) REFERENCES student(ID),
   FOREIGN KEY (sec_id, semester, year, course_id)
      REFERENCES section(sec_id, semester, year, course_id)
);
```

Sample Data

sample_data.sql

```
-- Department data
INSERT INTO department VALUES ('CS', 'Taylor', 950000.00);
INSERT INTO department VALUES ('BIO', 'Watson', 875000.00);

-- Student data
INSERT INTO student VALUES ('S1001', 'John Smith', 90, 'CS');
INSERT INTO student VALUES ('S1002', 'Maria Rodriguez', 85, 'CS');

-- Instructor data
INSERT INTO instructor VALUES ('I101', 'Dr. Jane Doe', 85000.00, 'CS');

-- Course data
INSERT INTO course VALUES ('CS101', 'Introduction to Programming', 3, 'CS');

-- Takes relationship (IMPORTANT)
INSERT INTO takes VALUES ('S1001', '1', 'Fall', 2024, 'CS101', 'A');

-- Teaches relationship
INSERT INTO teaches VALUES ('I101', '1', 'Fall', 2024, 'CS101');
```

Java Source Code

1

__SHELL0.java

```
University.UniversityDatabaseApp.main(__bluej_param0);
```

2

CompactFormatte
r.java

```
ResultSetMetaData metaData = resultSet.getMetaData();
int columnCount = metaData.getColumnCount();

// Get column names and determine their display width
for (int i = 1; i <= columnCount; i++) {
   columnNames[i-1] = metaData.getColumnLabel(i);
   columnWidths[i-1] = Math.max(columnNames[i-1].length(), 5);
}

// First pass: determine column widths based on data
resultSet.beforeFirst();
while (resultSet.next()) {
   for (int i = 1; i <= columnCount; i++) {
      String value = resultSet.getString(i);
      if (value == null) value = "NULL";
      columnWidths[i-1] = Math.max(columnWidths[i-1], value.length());
   }
}
```

3

DatabaseConnect
ion.java

```
connection = DriverManager.getConnection(
   URL + DATABASE_NAME,
   USER,
   PASSWORD
);

// If database doesn't exist, create it
if (e.getMessage().contains("Unknown database")) {
   Connection tempConn = DriverManager.getConnection(URL, USER, PASSWORD);
   tempConn.createStatement().executeUpdate("CREATE DATABASE " + DATABASE_NAME);
   tempConn.close();

   connection = DriverManager.getConnection(
      URL + DATABASE_NAME,
      USER,
      PASSWORD
   );
}
```

4

InstructorManager.java

```
String sql = "INSERT INTO instructor VALUES (?, ?, ?, ?)";

try (PreparedStatement stmt = connection.prepareStatement(sql)) {
   stmt.setString(1, id);
   stmt.setString(2, name);
   stmt.setDouble(3, Double.parseDouble(salary));

   if (deptName == null) {
      stmt.setNull(4, Types.VARCHAR);
   } else {
      stmt.setString(4, deptName);
   }

   int rowsAffected = stmt.executeUpdate();
   if (rowsAffected > 0) {
      System.out.println("Instructor inserted successfully!");
   }
}
```

5

RecordManager.java

```
String sql = "INSERT INTO section (course_id, sec_id, semester, year) VALUES (?, ?, ?, ?)";

try (PreparedStatement stmt = connection.prepareStatement(sql)) {
   stmt.setString(1, courseId);
   stmt.setString(2, secId);
   stmt.setString(3, semester);
   stmt.setInt(4, Integer.parseInt(year));

   int rowsAffected = stmt.executeUpdate();
   if (rowsAffected > 0) {
      System.out.println("Section record inserted successfully!");
   }
}
```

6

ReportGenerator.java

```
String sql = "SELECT t.semester, t.year, s.ID, s.name, t.course_id, c.title, t.grade " +
        "FROM student s JOIN takes t ON s.ID = t.student_id " +
        "JOIN course c ON t.course_id = c.course_id " +
        "JOIN (" +
        "   SELECT semester, year, MAX(grade) as max_grade " +
        "   FROM takes WHERE grade IS NOT NULL " +
        "   GROUP BY semester, year" +
        ") m ON t.semester = m.semester AND t.year = m.year AND t.grade = m.max_grade";
```

7

SearchManager.java

```
private void storeSearchResults(ResultSet resultSet) throws SQLException {
  if (resultSet == null) {
    lastSearchResults = null;
    lastSearchColumnNames = null;
    return;
  }

  ResultSetMetaData metaData = resultSet.getMetaData();
  int columnCount = metaData.getColumnCount();

  // Store column names
  lastSearchColumnNames = new ArrayList<>();
  for (int i = 1; i <= columnCount; i++) {
    lastSearchColumnNames.add(metaData.getColumnLabel(i));
  }

  // Store data rows
  lastSearchResults = new ArrayList<>();
  while (resultSet.next()) {
    List<String> row = new ArrayList<>();
    for (int i = 1; i <= columnCount; i++) {
      row.add(resultSet.getString(i));
    }
    lastSearchResults.add(row);
  }

  // Reset cursor for reuse
  resultSet.beforeFirst();
}
```

8


```
String sql = "INSERT INTO student VALUES (?, ?, ?, ?)";
try (PreparedStatement stmt = connection.prepareStatement(sql)) {
   stmt.setString(1, id);
   stmt.setString(2, name);

   if (deptName == null) {
      stmt.setNull(3, Types.VARCHAR);
   } else {
      stmt.setString(3, deptName);
   }

   stmt.setInt(4, Integer.parseInt(totCred));

   int rowsAffected = stmt.executeUpdate();
   if (rowsAffected > 0) {
      System.out.println("Student inserted successfully!");
   } else {
      System.out.println("Failed to insert student.");
   }
}
```

9


```
private void initializeManagers(Connection connection) {
   studentManager = new StudentManager(connection);
   instructorManager = new InstructorManager(connection);
   reportGenerator = new ReportGenerator(connection);
   recordManager = new RecordManager(connection);
   searchManager = new SearchManager(connection);
}
```

Brief User Guide

1. Run the Application
   Execute the main class (UniversityDatabaseApp) using an IDE such as BlueJ.
2. Database Setup
   Ensure MySQL server is running, and the database is configured correctly.
3. Menu Options
   The application provides options such as:
   - Add Student
   - View Records
   - Update Data
   - Delete Data
   - Search and Reports
4. User Input
   Follow on-screen instructions to enter required information.
5. Error Handling
   The system validates user input and ensures proper database operations.

GitHub link https://github.com/Abdullah-Tauqir/CS1103-Database-Project.git